

Doc. No.: WG21/P0250R1
Date: 2016-03-20
Author: Hans-J. Boehm
Email: hboehm@google.com

Target audience: SG1 and CWG

P0250R1: Wording improvements for initialization and thread ids (CWG 2046)

This is an attempt to turn the Kona SG1 discussion of CWG 2046 into wording. This revision also reflects discussion in Jacksonville. Unfortunately, these discussions exposed a number of related issues that are still not fully resolved.

Happens-before constraints on the implementation

In several places the standard needs to require that some operation, e.g. an initialization, happens before another one, in roughly, but not quite, the sense of the 1.10 memory model. The difference is that we expect this happens before relation to compose with other happens-before relationships. But our normal happens before relation no longer composes in this way, due to the presence of `memory_order_consume`. Hence we suggest introducing the required notion in 1.10.

There was some discussion in Jacksonville about using "synchronizes with" instead of introducing new terminology. That appears to be technically workable, but rather misleading. "Synchronizes with" is currently only defined between actions by different threads, and does not include sequencing. Many of the guarantees we would like to provide will often rely on sequencing rather than synchronization. Thus my current feeling is to introduce new terminology.

The current standard sometimes uses "sequenced before" to express this kind of relationship. This is incorrect and repaired below, since sequencing is a relation on evaluations within a single thread.

Parallel initialization for sequential programs

In Kona, SG1 rather quickly concluded that a C++ program should be allowed to construct multiple threads for constructors etc., even if the program itself did not use threads. That is not reflected here, mostly because it is a more profound issue than we appreciated at the time.

If we went this route, we would break compatibility with C++03, in what may be a rather significant way. Static constructors for old single-threaded programs would be required to be thread-safe. This is unlikely to be automatically true. It is not that uncommon for constructors to e.g. add to a global data structure. The order in which items are added may not matter. But such a data structure implemented in a single-threaded program is unlikely to be thread-safe.

In Jacksonville, it became clear that some of the confusion here stems from uncertainty about the motivation for the current wording. Currently, we very explicitly allow static constructors to run after the start of main, whether or not other threads are started. Running a constructor while main is running inherently implies some notion of concurrency. The intent appears to have been to allow construction right before the first odr-use within the using thread, rather than at an arbitrary point before the initial odr-use, so that unexpected races between the constructor and main can be avoided. We don't yet know how to phrase this.

The Jacksonville discussion seemed to lean towards preserving wording that allows parallel initialization only after a second thread is forked. However it remains unclear what this means, given that I don't think we're currently very clear that random library calls cannot start threads. We would probably need a similar restriction prohibiting under-the-covers thread creation by library calls unless a second thread has already been created.

Thread id for main

We concluded in Kona that main did not necessarily have a thread id, and that some implementations went out of their way to keep it from having one. After looking at the N4567 wording again, my conclusion is that this interpretation of the standard is a stretch. Some of this wording may have improved since the implementers looked at it.

The first sentence of 1.10 [intro.multithread] states

A thread of execution (also known as a thread) is a single flow of control within a program, including the initial invocation of a specific top-level function, and recursively including every function invocation subsequently executed

by the thread.

30.3.1.1 [thread.thread.id] states

An object of type `thread::id` provides a unique identifier for each thread of execution and a single distinct value for all thread objects that do not represent a thread of execution (30.3.1).

30.3.2 [thread.thread.this] states

`thread::id this_thread::get_id()` noexcept;

Returns: An object of type `thread::id` that uniquely identifies the current thread of execution. No other thread of execution shall have this id and this thread of execution shall always have this id. The object returned shall not compare equal to a default constructed `thread::id`.

I would nonetheless suggest changing the first sentence of 3.6.1 [basic.start.main] as follows:

A program shall contain a global function called `main`, which is the designated start of the program and of its initial thread of execution.

There were some voices in Jacksonville in favor of preparing the standard for execution agents that do not support a thread id (or thread local storage). This would simplify GPU execution, but it seems to require that the standard library specify which calls need thread local storage, and which do not, something we don't currently specify. It is unclear to me that we should address that issue in this context.

Wording changes for construction/destruction ordering

(Fixes an editorial ugliness that somehow crept in.) Move 1.9p6, which uses sequenced before, to someplace below the definition of sequenced before in 1.9p13 [intro.execution].

Add after 1.10p14 [intro.multithread]:

An evaluation A strongly happens before an evaluation B if either

- A is sequenced before B, or
- A synchronizes with B, or
- A strongly happens before X and X strongly happens before B.

[Note: In the absence of consume operations the happens before and strong happens before relations are identical. Strong happens before essentially excludes consume operations.]

There may be cleaner ways to introduce this.

Change the last part of 3.6.2p2 [basic.start.static] as follows:

If constant initialization is not performed, a variable with static storage duration (3.7.1) or thread storage duration (3.7.2) is zero-initialized (8.5). Together, zero-initialization and constant initialization are called static initialization; all other initialization is dynamic initialization. Within each phase, static initialization shall ~~be performed~~ strongly happen before (1.10) any dynamic initialization ~~takes place~~. [Note: The dynamic initialization of non-local variables is described in 3.6.3; that of local static variables is described in 6.7. -- end note]

Change 3.6.3p1 [basic.start.dynamic] as follows:

Dynamic initialization of a non-local variable with static storage duration is unordered if the variable is an implicitly or explicitly instantiated specialization, and otherwise is ordered [Note: an explicitly specialized static data member or variable template specialization has ordered initialization. -- end note]. Variables with ordered initialization defined within a single translation unit shall be initialized ~~in the order of their definitions~~ such that initialization corresponding to earlier definitions strongly happen before those for later definitions in the translation unit. If a program starts a thread (30.3), any initialization not specified to happen before the first thread creation may be performed by either the main thread or a thread implicitly created to perform initializations. If such initializations are unordered or defined in separate translation units, no happens before ordering is specified between them. the subsequent initialization of a variable is unsequenced with respect to the initialization of a variable defined in a different translation unit. Otherwise, the initialization of a variable is indeterminately sequenced with respect to the initialization of a variable defined in a different translation unit. If a program starts a thread, the subsequent unordered initialization of a variable is unsequenced with respect to every other dynamic initialization. Otherwise, If two unordered initializations happen before the start of the first additional thread, or the program starts no

~~additional threads, then those the unordered initializations of a variable is~~ are indeterminately sequenced with respect to each other every other dynamic initialization. [Note: This definition permits initialization of a sequence of ordered variables concurrently with another sequence. -- end note]

3.6.3p2 appears to have some unfortunate interactions with signal handlers. We may want to address those here, but they are currently not addressed. P0270R0 has more details.

Change the beginning of 3.6.3p2 as follows:

It is implementation-defined whether the dynamic initialization of a non-local variable with static storage duration strongly happens before the first statement of main. If the initialization is deferred to some point in time after the first statement of main, it strongly happens before the first odr-use (3.2) of any function or variable defined in the same translation unit as the variable to be initialized.

As we discussed above, this almost certainly needs to be pinned down further. Can we say that it happens immediately before the first odr-use, i.e. after all evaluations sequenced before the odr-use? Can we just insist that it must happen before main, and drop the second option?

Change 3.6.3p3 as follows to fix what looks like an unrelated bug:

It is implementation-defined whether the dynamic initialization of a non-local variable with ~~static or~~ thread storage duration is sequenced before the first statement of the initial function of the thread. If the initialization is deferred to some point in time after the first statement of the initial function of the thread, it is sequenced before the first odr-use (3.2) of any variable with thread storage duration defined in the same translation unit as the variable to be initialized.

The preceding seems to have similar issues to static variables, in that it allows essentially concurrent initialization of thread-locals. If a thread-local constructor touches a data structure also touched by the mainline thread code, this allows concurrent access from the same thread, which seems highly undesirable and unexpected. This presumably needs clarification similar to the static case.

Change 3.6.4p1 [basic.start.term] as follows:

Destructors (12.4) for initialized objects (that is, objects whose lifetime (3.8) has begun) with static storage duration are called as a result of returning from main and as a result of calling `std::exit` (18.5). The return from main or the call to `std::exit` respectively strongly happen before the destructor invocations.

Destructors for initialized objects with thread storage duration within a given thread are called as a result of returning from the initial function of that thread and as a result of that thread calling `std::exit`. The completions of the destructors for all initialized objects with thread storage duration within that thread ~~are sequenced~~ strongly happen before the initiation of the destructors of any object with static storage duration. If the completion of the constructor or dynamic initialization of an object with thread storage duration is sequenced before that of another, the completion of the destructor of the second is sequenced before the initiation of the destructor of the first.

If the completion of the constructor or dynamic initialization of an object with static storage duration ~~is sequenced~~ strongly happens before that of another, the completion of the destructor of the second ~~is sequenced~~ strongly happens before the initiation of the destructor of the first. [Note: This definition permits concurrent destruction. -- end note] If an object is initialized statically, the object is destroyed in the same order as if the object was dynamically initialized. For an object of array or class type, all subobjects of that object are destroyed before any block-scope object with static storage duration initialized during the construction of the subobjects is destroyed. If the destruction of an object with static or thread storage duration exits via an exception, `std::terminate` is called (15.5.1).

Change 3.6.4p3 [basic.start.term] as follows:

If the completion of the initialization of an object with static storage duration ~~is sequenced~~ strongly happens before a call to `std::atexit` (see <cstdlib>, 18.5), the call to the function passed to `std::atexit` ~~is sequenced~~ strongly happens before the call to the destructor for the object. If a call to `std::atexit` ~~is sequenced~~ strongly happens before the completion of the initialization of an object with static storage duration, the call to the destructor for the object ~~is sequenced~~ strongly happens before the call to the function passed to `std::atexit`. If a call to `std::atexit` ~~is sequenced~~ strongly happens before another call to `std::atexit`, the call to the function passed to the second `std::atexit` call ~~is sequenced~~ strongly happens before the call to the function passed to the first `std::atexit` call.